

SOFTWARE REQUIREMENTS SPECIFICATION

Aesthetica

Full-Featured E-Commerce Web Application

Java 17 | Hibernate 6 | Jersey 3 | MySQL 5.7+

Version 1.0

Prepared by	Dehemi Fabio
Document Type	Software Requirements Specification
Version	1.0
Date	May 29, 2026
Status	Done
Tech Stack	Java 17, Maven, Apache Tomcat 10.1.7, Jersey 3.1.2, Hibernate 6.1.7, MySQL 5.7+

1. Introduction

Aesthetica is a full-featured e-commerce web application built using modern Java technologies. It provides a comprehensive marketplace platform where users can browse products, manage shopping carts, place orders, and handle payments through PayHere integration. This SRS document formally defines the software requirements for the Aesthetica platform and serves as the authoritative reference for design, development, and testing activities.

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the Aesthetica e-commerce platform. It is intended for use by developers, testers, project stakeholders, and any other parties involved in the delivery and maintenance of the system. The document follows the SRS template of the Java Institute for Advanced Technology and covers the full scope of the application including its layered architecture, RESTful API design, database schema, entity relationships, and user interactions.

1.2 Product Scope

Aesthetica is a web-based multi-seller marketplace. Customers can discover and purchase products; sellers can manage their listings; and administrators oversee the entire platform. The primary benefits of the system include:

- A seamless, responsive product catalogue with category-based browsing, keyword search, and inventory-aware pricing
- A secure shopping cart and checkout experience integrated with the PayHere payment gateway
- Multi-seller support allowing independent vendors to list and manage their products
- Flexible delivery options (within-city at LKR 300 and out-of-city at LKR 500) with coupon/discount support
- Email notifications for registration confirmation and order updates via Jakarta Mail
- A comprehensive user account dashboard for profile management, address management, and order history

1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
ORM	Object-Relational Mapping — Hibernate maps Java entities to MySQL tables
DTO	Data Transfer Object — decouples API layer from entity/database layer
REST	Representational State Transfer — architectural style for HTTP APIs
JAX-RS	Jakarta RESTful Web Services specification, implemented by Jersey

JPA	Jakarta Persistence API — standard for ORM with Hibernate as provider
WAR	Web Application Archive — packaged deployment artifact
HTTPS	Hypertext Transfer Protocol Secure
PCI-DSS	Payment Card Industry Data Security Standard
CDN	Content Delivery Network
GDPR	General Data Protection Regulation
PayHere	Sri Lankan payment gateway used for transaction processing
H2	In-memory relational database used as fallback during development/testing
Admin	System administrator with elevated access privileges
Seller	Registered vendor who lists products on the marketplace
Customer	Registered or guest end user who browses and purchases products

1.4 References and Acknowledgments

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Java Institute for Advanced Technology — SRS Template v1.0
- Jakarta EE 10 / Jersey 3.1.2 documentation — JAX-RS REST framework
- Hibernate ORM 6.1.7 documentation — object-relational mapping
- PayHere Developer Documentation — Merchant ID: 1224621
- OWASP Web Security Testing Guide — security implementation reference

2. Overall Description

2.1 Software Perspective

Aesthetica is a standalone web application following a Layered N-Tier Architecture with clean separation of concerns. The application runs on an embedded Apache Tomcat 10.1.7 server, enabling standalone deployment without external server configuration. The six architectural layers are:

Layer	Technology	Responsibility
Presentation Layer	HTML5, CSS3 (Bootstrap), JavaScript ES6+	User-facing pages: index, shop, product detail, checkout, account, contact
REST API Layer	Jersey 3.1.2 (JAX-RS)	Controllers expose RESTful endpoints; map HTTP requests to DTOs
Middleware & Filters	Jakarta Servlet Filters	AuthAccessFilter, AccessControlFilter, AuthFilter; @IsUser annotation
Business Logic Layer	Java 17 Services	Validation, business rules, payment flow orchestration
Data Access Layer	Hibernate ORM 6.1.7	Maps Java entities to MySQL tables; named queries; lazy loading
Database Layer	MySQL 5.7+ (H2 fallback)	Persistent storage of users, products, orders, carts, etc.

External integrations include the PayHere payment gateway for transaction processing and Jakarta Mail (Mailtrap for test, SMTP for production) for transactional email delivery.

2.2 Users and Characteristics

2.2.1 Customers (Buyers)

Primary end users of the marketplace storefront. They register/login, browse the product catalogue, add items to their shopping cart, complete checkout via PayHere, and track orders from their account dashboard. Moderate familiarity with web-based e-commerce platforms is assumed. Authentication is managed via HTTP sessions.

2.2.2 Sellers

Registered vendors who list products on the platform. A Seller entity is linked one-to-one with a User entity and has an associated business address, company name, mobile, and email. Sellers are subject to approval (status: PENDING → APPROVED / REJECTED) by administrators before their products are visible.

2.2.3 Administrators

Internal staff with elevated system access responsible for platform oversight, content management, user account management, order monitoring, and seller approval. Administrators interact via the admin panel and are assumed to have working knowledge of content and order management workflows.

2.3 Operating Environment

- Server OS: Linux-based server environment
- Web Server: Apache Tomcat 10.1.7 (embedded, standalone JAR/WAR deployment)
- Backend Language: Java 17 with Maven 3.x build tooling
- Database: MySQL 5.7+ (primary); H2 in-memory database (development/testing fallback)
- Database URL: jdbc:mysql://localhost:3306/aesthetica
- Client: Modern web browsers — Chrome, Firefox, Safari, Edge (latest two major versions)
- Frontend: HTML5, CSS3 with Bootstrap, JavaScript ES6+; UI libraries: Notiflix (notifications), noUiSlider (price range filter)
- Email: Jakarta Mail 2.0.1+ with Rocketbase Email Template builder; Mailtrap SMTP for testing
- Deployment Artifact: aesthetica.war packaged via Maven

2.4 Design and Implementation Constraints

- The system must use Java 17 and Maven 3.x as the build toolchain
- REST endpoints must be implemented using Jersey 3.1.2 (JAX-RS); all API responses are in JSON format via GSON 2.10.1
- All database operations must go through the Hibernate ORM layer; direct JDBC is not permitted in business logic
- hbm2ddl.auto=update is used; schema migrations are managed via SQL migration files (db_migration_2026_02_20.sql)
- Payment processing is exclusively handled through the PayHere gateway (Merchant ID: 1224621); card data must never be stored on the application server
- All API endpoints requiring authentication must be protected with the `@IsUser` annotation and validated by `AuthFilter`
- Password storage must use a secure hashing approach; plain-text passwords are not permitted
- All server-client communication must use HTTPS in production

2.5 Assumptions and Dependencies

- A MySQL 5.7+ database instance is available at localhost:3306 with schema 'aesthetica'; if unavailable, the system falls back to H2 in-memory database automatically via `HibernateUtil`
- PayHere merchant credentials (ID and secret) are correctly configured in `app.properties` for payment processing

- SMTP credentials (Mailtrap or production) are configured in app.properties for email delivery
- Users have access to a stable internet connection and a compatible modern web browser
- Product images are stored on the server filesystem and served via the web application; image paths are stored as an ElementCollection in the product_images table
- Cities, delivery types, status values, and default discount (0%) are pre-seeded via db_seed_data.sql before the application is first launched

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Aesthetica platform consists of the following HTML pages, each with a corresponding JavaScript file handling event listeners, form validation, API calls (fetch), DOM manipulation, and Notiflix notifications:

Page File	Title	Description
index.html	Home	Hero banner, featured products, category navigation, footer
shop.html	Product Listing	Product grid with noUiSlider price filter, category filter, keyword search, pagination
single-product-view.html	Product Detail	Full product images, description, available stock/price variants, seller info, ratings, add-to-cart
checkout.html	Checkout	Cart review, delivery address selection, delivery type selection, discount code entry, PayHere payment
my_account.html	My Account	Personal info, address management, order history with status
sign_up.html	Register	Registration form: firstName, lastName, email, mobile, password, confirmPassword
sign_in.html	Login	Login form: email, password; redirect to home on success
all_categories.html	Categories	Grid of all product categories with icons
contact.html	Contact	Support/contact form

3.1.2 Hardware Interfaces

The system does not interface directly with specialised hardware. Standard input devices (keyboard, mouse, touchscreen) are supported through the browser layer. The server hardware must support Java 17 runtime and MySQL 5.7+. Image assets are stored on the server filesystem and accessed via standard file I/O (Commons IO 2.16.1).

3.1.3 Software Interfaces

Interface	Technology / Version	Purpose
Database	MySQL 5.7+ via MySQL Connector 9.0.0	Persistent storage; Hibernate dialect: org.hibernate.dialect.MySQLDialect
ORM	Hibernate 6.1.7.Final	Entity mapping, session management, named queries, lazy loading, auto schema update

REST Framework	Jersey 3.1.2 (JAX-RS)	RESTful HTTP API; JSON via GSON 2.10.1; configured in AppConfig.java
Payment Gateway	PayHere REST API (PayHereUtil.java)	Payment initiation and callback handling; merchant credentials in app.properties
Email Service	Jakarta Mail 2.0.1 + Rocketbase Template 2.4.0	Transactional emails: registration, order confirmation; configured via Env.java
Servlet Container	Apache Tomcat 10.1.7 (embedded)	HTTP request handling; session management; filter chain
Validation	Hibernate Validator 8.0.0.Final	Bean validation at API boundary
Dev DB Fallback	H2 In-Memory Database	Automatic fallback if MySQL is unavailable at startup (HibernateUtil)

3.2 Functional Requirements

3.2.1 User Management (UserController — /api/users)

ID	Requirement	Description	Auth
FR-01	User Registration	POST /api/users: Accepts {firstName, lastName, email, password, confirmPassword, mobile}. Validates email uniqueness (Named Query), password strength (min 8 chars, 1 uppercase, 1 lowercase, 1 digit, 1 special char), and mobile format (10-digit Sri Lankan). Sets Status to ACTIVE. Returns {status, message, userId}.	Public
FR-02	User Login	POST /api/users/login: Accepts {email, password}. Queries user by email, compares password. Creates HTTP session storing User object. Returns UserDTO on success.	Public
FR-03	User Logout	GET /api/users/logout: Invalidates current HTTP session. Returns 202 ACCEPTED or 400 BAD_REQUEST.	@IsUser
FR-04	Auth Redirect	AuthAccessFilter redirects already-authenticated users away from /sign_up.html and /sign_in.html to home. AccessControlFilter redirects unauthenticated users away from /my_account.html and /checkout.html to login.	Filter

3.2.2 Product Management (ProductController — /api/products)

ID	Requirement	Description	Auth
FR-05	List Products	GET /api/products?page=1&size=10&category=1&search=term:	Public

		Returns paginated List<ProductDTO> with filtering by category and keyword search.	
FR-06	Get Product Detail	GET /api/products/{id}: Returns full ProductDTO including images, seller info, available stock variants with prices, quantities, and discount information.	Public
FR-07	List Categories	GET /api/products/categories: Returns all active product categories with name and icon.	Public

3.2.3 Shopping Cart (CartController — /api/cart)

ID	Requirement	Description	Auth
FR-08	Add to Cart	POST /api/cart: Accepts {stockId, quantity}. CartService validates stock availability before creating Cart entity linking User + Stock + Quantity.	@IsUser
FR-09	View Cart	GET /api/cart: Returns List<CartDTO> for the logged-in user including item prices, product titles, and quantities.	@IsUser
FR-10	Update Cart Item	PUT /api/cart/{cartId}: Accepts {quantity} to update the quantity of a specific cart item.	@IsUser
FR-11	Remove Cart Item	DELETE /api/cart/{cartId}: Removes a specific item from the user's cart.	@IsUser
FR-12	Clear Cart	DELETE /api/cart/clear: Removes all items from the user's cart.	@IsUser

3.2.4 Checkout & Payment (CheckoutController — /api/checkout)

ID	Requirement	Description	Auth
FR-13	Process Checkout	POST /api/checkout: Accepts CheckoutRequestDTO {deliveryTypeId, discountCode, addressId, paymentMethod, paymentDetails}. CheckoutService validates address, calculates total (items + delivery - discount), calls PayHereUtil.initiatePayment(), creates Order and OrderItems on success, clears cart, sends confirmation email.	@IsUser
FR-14	Payment Callback	Handles PayHere callback to /api/checkout/callback to confirm payment and finalise order creation.	Gateway
FR-15	Get Delivery Types	GET /api/checkout/delivery-types: Returns List<DeliveryTypeDTO> — WITHIN_CITY (LKR 300) and OUT_OF_CITY (LKR 500).	Public

3.2.5 User Profile (ProfileController — /api/profile)

ID	Requirement	Description	Auth
FR-16	Get Profile	GET /api/profile: Returns full UserDTO with profile information for the authenticated user.	@IsUser
FR-17	Update Profile	PUT /api/profile: Accepts {firstName, lastName, mobile, newPassword}. Updates user information and/or password.	@IsUser
FR-18	Add Address	POST /api/profile/address: Accepts UserAddressDTO {lineOne, lineTwo, postalCode, cityId, isPrimary}. Creates new delivery address linked to user.	@IsUser
FR-19	List Addresses	GET /api/profile/addresses: Returns all addresses for the authenticated user. The Named Query Address.getPrimary retrieves the user's primary address.	@IsUser
FR-20	Order History	GET /api/profile/orders: Returns paginated list of the user's orders with status, date, items, and totals.	@IsUser

3.2.6 Content / Reference Data (ContentController — /api/content)

ID	Requirement	Description	Auth
FR-21	Get Cities	GET /api/content/cities: Returns List<CityDTO> for delivery address city selection. Pre-seeded: Colombo, Kandy, Galle, Jaffna, Kurunegala, etc.	Public
FR-22	Get Categories	GET /api/content/categories: Returns List<CategoryDTO> with name and icon path for product filtering and browse.	Public
FR-23	Get Delivery Types	GET /api/content/delivery-types: Returns delivery options with pricing for frontend display.	Public

3.3 Behavior Requirements — Use Case View

The following primary actors are identified: Customer, Seller, and Administrator. Key use cases are described below.

UC ID	Actor	Use Case	Brief Description
UC-01	Customer	Register Account	User submits registration form (sign_up.html); sign_up.js validates locally then POST /api/users; on success redirect to sign_in.html.
UC-02	Customer	Login / Logout	User authenticates via POST /api/users/login creating an HTTP session; logout via GET /api/users/logout destroys

			session.
UC-03	Customer	Browse & Search Products	shop.html calls GET /api/products with optional category, search, and page parameters; results displayed in responsive grid.
UC-04	Customer	View Product Detail	single-product-view.html calls GET /api/products/{id}; displays images, stock variants, prices, seller info, and ratings.
UC-05	Customer	Manage Cart	Authenticated customer adds (POST), updates (PUT), removes (DELETE) items; cart totals calculated by CartService.
UC-06	Customer	Checkout & Pay	Customer selects address, delivery type, optional discount code; CheckoutService calculates total; payment processed via PayHere; order created; email sent; cart cleared.
UC-07	Customer	View Order History	Authenticated customer views orders with status (PENDING, PACKING, DELIVERED, COMPLETED, CANCELLED) from GET /api/profile/orders.
UC-08	Customer	Manage Profile & Addresses	Authenticated customer updates personal info/password via PUT /api/profile; manages delivery addresses via /api/profile/address.
UC-09	Seller	Manage Product Listings	Seller uploads products with images, category, dimensions, and multiple stock variants (price/quantity/expiry batches); subject to admin approval.
UC-10	Admin	Manage Users & Sellers	Admin reviews accounts, activates/deactivates/blocks users, approves or rejects seller applications, monitors orders.

4. Other Non-Functional Requirements

4.1 Performance Requirements

- Product listing pages must load within 3 seconds under normal network conditions
- Search and category filter queries must return results within 2 seconds for standard pagination sizes
- Hibernate lazy loading must be used by default for all entity relationships to prevent N+1 query issues
- Named queries (pre-compiled HQL) must be used for all frequently executed lookups (e.g., user by email, status by value, primary address by user)
- The system must support a minimum of 500 concurrent users; the embedded Tomcat thread pool must be configured accordingly
- Checkout and PayHere payment initiation must complete within 5 seconds under standard network conditions
- Database connection pooling is managed by Hibernate; the configuration must be tuned for expected concurrent sessions

4.2 Safety and Security Requirements

- All user passwords must be stored using a strong cryptographic hashing algorithm (e.g., bcrypt); plain-text passwords are strictly prohibited
- All client-server communication must be encrypted via HTTPS / TLS 1.2 or higher in the production environment
- Payment processing is exclusively handled through the PayHere gateway; card details must never be stored on Aesthetica application servers (PCI-DSS compliance)
- All API endpoints requiring authentication must be protected by the `@IsUser` annotation enforced by `AuthFilter`; unauthenticated requests must receive a 401 Unauthorized response
- The `AccessControlFilter` must redirect unauthenticated users attempting to access `/my_account.html` or `/checkout.html` to the login page
- Rate limiting must be applied to the login endpoint (POST `/api/users/login`) to mitigate brute-force attacks
- HTTP sessions must expire after a defined period of inactivity to prevent session hijacking
- Input validation is enforced at three layers: (1) frontend JavaScript, (2) service-layer business rules, and (3) database constraints (UNIQUE, NOT NULL, length limits)
- SQL injection is prevented by the Hibernate ORM layer; raw JDBC queries in business logic are not permitted
- User personal data must be collected, stored, and processed in accordance with applicable data protection regulations (GDPR)
- Merchant credentials (PayHere merchant ID and secret, SMTP credentials) must be stored in `app.properties` and never hard-coded in source files

Appendix A — Database Entity Summary

The following entities are mapped by Hibernate ORM to the MySQL 'aesthetica' database. All entities inheriting from BaseEntity include created_at and updated_at timestamp fields managed automatically by @CreationTimestamp.

Entity	Table	Key Fields	Key Relationships
User	users	id, firstName, lastName, email (unique), password (hashed), mobile (10-digit), status_id	OneToMany: Address, Cart, Order; Status FK
Product	product	id, title (200), description (TEXT), weight/length/width/height (default 1.0), seller_id (nullable), category_id	ManyToOne: Seller, Category; OneToMany: Stock; ElementCollection: images (product_images)
Stock	stock	id, product_id, price, quantity, manufactured_date, expiry_date, discount_id, status_id	ManyToOne: Product, Discount, Status
Cart	cart	id, user_id, stock_id, qty	ManyToOne: User, Stock
Order	orders	id, user_id, delivery_types_id, status_id, created_at, updated_at	ManyToOne: User, DeliveryType, Status; OneToMany: OrderItem
OrderItem	order_items	id, order_id, stocks_id, seller_id, quantity, rating (0-5)	ManyToOne: Order, Stock, Seller
Address	address	id, user_id, line_one, line_two, postalCode, city_id, is_primary	ManyToOne: User, City; Named Query: Address.getPrimary
Category	category	id, name, icon	OneToMany: Product
Seller	seller	id, company_name, company_mobile, company_email, user_id, address_id, status_id	OneToOne: User, Address; OneToMany: Product; ManyToOne: Status
Status	status	id, value (UNIQUE)	Values: ACTIVE, INACTIVE, BLOCKED, DEACTIVATE, VERIFIED, PENDING, PACKING, DELIVERED, COMPLETED, CANCELLED, APPROVED, REJECTED
DeliveryType	delivery_types	id, name, price	Values: WITHIN_CITY (LKR 300), OUT_OF_CITY (LKR 500)
Discount	discount	id, coupon_code (UNIQUE), value, started_at, expired_at	Default coupon: 'DEFAULT' at 0%
City	city	id, name	Pre-seeded: Colombo, Kandy, Galle, Jaffna, Kurunegala, etc.

Appendix B — Group Log

Date	Author	Description	Version
2026-05-29	Dehemi	Initial SRS drafted based on Aesthetica project documentation (Java 17, Hibernate 6, Jersey 3, MySQL 5.7+)	1.0

End of Document